

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Iulian Pavlov, FAF-241

Report

Laboratory work nr. 10

Add a System Call: `trace` (xv6)

Course: Operating Systems (Track B: build your own small OS (xv6))

Checked by:
Reiţman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Contents

1 Theoretical Background and Objectives **1**
1.1 Laboratory Objectives and Scope 1
1.2 System Call Path and Bitmask Architecture in xv6 1

2 Part 1: User-Space Syscall Plumbing and `trace` Utility **2**
2.1 Syscall Number, Prototype, Stub, and Build Integration 2
2.2 Implementing `user/trace.c` 2

3 Part 2: Kernel Implementation **3**
3.1 Extending `struct proc` and Inheriting Across `fork()` 3
3.2 System Call Handler `sys_trace` 3
3.3 Dispatch Table, Syscall Names, and Post-Call Logging 4

4 Part 3: QEMU Results and Observations **5**
4.1 Targeted Trace: `trace 32 grep hello README` 5
4.2 Full Bitmask Trace: `trace 2147483647 grep hello README` 5
4.3 Process Inheritance: `trace 2 usertests forkforkfork` 5
4.4 Console Output Interleaving: `trace 65536 echo hi` 7
4.5 Answers to Observation Questions (`notes.txt`) 7

5 Conclusions **8**

1 Theoretical Background and Objectives

1.1 Laboratory Objectives and Scope

In Laboratory Work No. 9, system calls were observed from user space on Linux using `strace` and the external `ptrace()` debugging interface. In Laboratory Work No. 10, we move inside the kernel of our teaching operating system, **xv6-riscv**, to design and implement a native tracing facility from scratch: a new system call `trace(int mask)` alongside a user-space command `trace mask command [args...]`.

The core objectives of Laboratory Work No. 10 are:

- **User-Space Syscall Plumbing**: Registering the new syscall number (`SYS_trace` = 23), declaring its C prototype in `user/user.h`, generating its RISC-V trap stub in `user/usys.pl`, writing `user/trace.c`, and registering it under `UPROGS` in `Makefile`.
- **Kernel Dispatch and Process State**: Adding `trace_mask` to `struct proc`, implementing `sys_trace()` in `kernel/sysproc.c`, registering `syscallnames[]` and `SYS_trace` in `kernel/syscall.c`, and logging matches via `printk()`.
- **Mask Inheritance Across Processes**: Propagating `trace_mask` from parent to child in `kfork()` (`kernel/proc.c`) while preserving it across `exec()`.
- **QEMU Verification and Behavioral Analysis**: Tracing `grep hello README`, `usertests forkforkfork`, and `echo hi` under selective and full bitmasks, explaining return codes, process inheritance, and console interleaving.

1.2 System Call Path and Bitmask Architecture in xv6

Every system call in xv6-riscv follows a fixed hardware and software sequence across the user–kernel privilege boundary:

1. **RISC-V User Stub**: Calling `trace(mask)` in C jumps to the assembly stub emitted by `user/usys.pl`, which leaves the first argument (`mask`) in register `a0`, loads `SYS_trace` into register `a7`, and executes `ecall`.
2. **Trap Entry and Dispatch**: The CPU enters supervisor mode, saves user registers in `p->trapframe`, and invokes `syscall()` inside `kernel/syscall.c`, reading `num` from `p->trapframe->a7`, running `syscalls[num]()`, and saving the result in `p->trapframe->a0`.
3. **Post-Call Bitmask Check**: Right after `syscalls[num]()` returns, `syscall()` tests bit `num` using `(p->trace_mask >> num) & 1`. If set, `printk()` emits the PID, name from `syscallnames[num]`, and return value from `p->trapframe->a0`.

Table 1: Selected System Call Constants (`kernel/syscall.h`) and Corresponding Trace Masks

Constant	ID	Decimal (2^{ID})	Hex Mask	Usage / Observation in Lab 10
`SYS_fork`	1	2	`0x00000002`	Fork storm trace in `forkforkfork`
`SYS_read`	5	32	`0x00000020`	Selective read trace in `grep hello README`
`SYS_exec`	7	128	`0x00000080`	Traced during full mask (`-> argc`)
`SYS_open`	15	32768	`0x00008000`	Combined with `close` in Q4 (`2129920`)
`SYS_write`	16	65536	`0x00010000`	Console write ordering in `echo hi`
`SYS_close`	21	2097152	`0x00200000`	Combined with `open` in Q4 (`2129920`)
`SYS_trace`	23	8388608	`0x00800000`	Self-traced under full mask `2147483647`
All calls	–	`2147483647`	`0x7fffffff`	Enables bits 0–30 (every syscall 1–23)

2 Part 1: User-Space Syscall Plumbing and `trace` Utility

2.1 Syscall Number, Prototype, Stub, and Build Integration

To expose `trace()` to xv6 user programs, four user-space and shared header edits were applied in `~/xv6-riscv`:

- **`kernel/syscall.h` (line 24)**: Assigned the next sequential syscall identifier right after `SYS_sync`:

```
#define SYS_sync   22
#define SYS_trace 23
```

- **`user/user.h` (line 28)**: Declared `int trace(int);` alongside the other system call declarations after `int sync(void);`.
- **`user/usys.pl` (line 46)**: Appended `entry("trace");`. During `make`, Perl emits the RISC-V assembly stub `.global trace`, `li a7, SYS_trace`, `ecall`, `ret` into `user/usys.S`.
- **`Makefile` (line 153)**: Registered `$U/_trace\` in `UPROGS` right after `$U/_zombie\` (preserving the required leading TAB character) so `mkfs` packages `/trace` into `fs.img`.

2.2 Implementing `user/trace.c`

The user-space wrapper `user/trace.c` parses the decimal bitmask from `argv[1]`, activates tracing on its own process via `trace(atoi(argv[1]))`, and replaces its process image with the target command using `exec(argv[2], argv + 2)`. Because `exec()` reuses

the caller's PID and `struct proc`, the newly loaded program runs with the requested `trace_mask` already armed.

Listing 1: User-space wrapper `user/trace.c`

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

Before adding the kernel handler in Part 2, invoking `trace 32 grep hello README` hits the fallback `else` branch in `syscall()`, printing `unknown sys call 23` and returning `-1` to user space (causing `trace: failed`). This serves as Check 1 confirming that the RISC-V `ecall` stub delivers syscall number `23` to the kernel.

3 Part 2: Kernel Implementation

3.1 Extending `struct proc` and Inheriting Across `fork()`

Each process stores its active bitmask inside its `struct proc` descriptor in `kernel/proc.h` (line 104):

```c
  char name[16];               // Process name (debugging)
  int trace_mask;              // which calls to print
```

To ensure that child processes spawned by a traced parent continue generating trace output, `kfork()` in `kernel/proc.c` (lines 290–291) copies the parent's `trace_mask` into the child's `struct proc`:

```c
  safestrcpy(np->name, p->name, sizeof(p->name));
  np->trace_mask = p->trace_mask;
```

Because the interactive shell runs with `trace_mask = 0`, any unrelated command forked directly by `sh` still inherits `0` and remains quiet unless explicitly launched through `/trace`.

3.2 System Call Handler `sys_trace`

In `kernel/sysproc.c` (lines 114–121), `sys_trace()` fetches the integer mask argument from trapframe register `a0` via `argint(0, &mask)` and saves it in the calling process structure:

Listing 2: Kernel handler `sys_trace` in `kernel/sysproc.c`

```
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

3.3 Dispatch Table, Syscall Names, and Post-Call Logging

In `kernel/syscall.c` (lines 106–168), we declared `extern uint64 sys_trace(void);`, added `[SYS_trace] = sys_trace` to `syscalls[]`, populated the `syscallnames[]` array, and inserted the conditional `printk()` call right after `syscalls[num]()` saves its return value in `p->trapframe->a0`:

Listing 3: Syscall name table and dispatch logging in `kernel/syscall.c`

```
static char *syscallnames[] = {
  [SYS_fork] "fork",   [SYS_exit] "exit",     [SYS_wait] "wait",
  [SYS_pipe] "pipe",   [SYS_read] "read",     [SYS_kill] "kill",
  [SYS_exec] "exec",   [SYS_fstat] "fstat",   [SYS_chdir] "chdir",
  [SYS_dup] "dup",     [SYS_getpid] "getpid", [SYS_sbrk] "sbrk",
  [SYS_pause] "pause", [SYS_uptime] "uptime", [SYS_open] "open",
  [SYS_write] "write", [SYS_mknod] "mknod",   [SYS_unlink] "unlink",
  [SYS_link] "link",   [SYS_mkdir] "mkdir",   [SYS_close] "close",
  [SYS_sync] "sync",   [SYS_trace] "trace",
};

void
syscall(void)
{
  int num;
  struct proc *p = myproc();

  num = p->trapframe->a7;
  if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    p->trapframe->a0 = syscalls[num]();
    if ((p->trace_mask >> num) & 1)
      printk("%d: syscall %s -> %ld\n", p->pid, syscallnames[num],
             p->trapframe->a0);
  } else {
    printk("%d %s: unknown sys call %d\n", p->pid, p->name, num);
    p->trapframe->a0 = -1;
  }
}
```

Two implementation details verified in `kernel/printk.c` matter for correctness:

- **Kernel console helper**: This xv6 tree defines `printk()` inside `kernel/printk.c` (lines 70–109) rather than a kernel `printf()`.
- **Signed 64-bit formatting (`%ld`)**: Register `p->trapframe->a0` holds a 64-bit `uint64`. Specifying `%ld` invokes signed decimal formatting (`printint(..., 10, 1)`), ensuring errors such as `-1` display cleanly as `-1` instead of an unsigned 64-bit integer.

4 Part 3: QEMU Results and Observations

4.1 Targeted Trace: `trace 32 grep hello README`

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

Mask `32` (2^5) sets only bit 5, corresponding to `SYS_read`. Process PID 3 (`grep`) reads `README` into its 1024-byte buffer (`buf[1024]` in `user/grep.c`, requesting `sizeof(buf) - m - 1` bytes per call where `m` is the length of any trailing incomplete line shifted forward). Across three non-empty chunks, it consumes $1023 + 965 + 453 = 2441$ bytes, followed by `0` indicating end-of-file.

4.2 Full Bitmask Trace: `trace 2147483647 grep hello README`

```
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open -> 3
4: syscall read -> 1023
4: syscall read -> 965
4: syscall read -> 453
4: syscall read -> 0
4: syscall close -> 0
```

Mask `2147483647` (`0x7fffffff`) enables bits 0 through 30, tracing every implemented syscall of PID 4:

1. `4: syscall trace -> 0`: Once `sys_trace()` sets mask `2147483647`, `syscall()` inspects bit 23 (`SYS_trace`) on return and logs `trace` returning `0`.
2. `4: syscall exec -> 3`: Replacing the image with `grep hello README` succeeds and places `argc` = `3` into `p->trapframe->a0`.
3. `4: syscall open -> 3`: Opening `README` allocates file descriptor `3`, since descriptors `0`, `1`, and `2` are already bound to standard input, output, and error.
4. `read` and `close -> 0`: After reading `README` to EOF, `grep` closes descriptor 3. Note that `exit` is absent because `sys_exit()` terminates the process and never returns past line 160 (`p->trapframe->a0 = syscalls[num]();`) in `kernel/syscall.c`.

4.3 Process Inheritance: `trace 2 usertests forkforkfork`

Running `usertests forkforkfork` with bitmask `2` (2^1, `SYS_fork`) verifies both mask propagation through `kfork()` and signed `-1` formatting once the kernel process table reaches `NPROC = 64` (`kernel/param.h`):

```
$ trace 2 usertests forkforkfork
usertests starting
test forkforkfork: 5: syscall fork -> 6
6: syscall fork -> 7
```

```
7: syscall fork -> 8
7: syscall fork -> 9
7: syscall fork -> 10
8: syscall fork -> 11
7: syscall fork -> 12
7: syscall fork -> 13
7: syscall fork -> 14
8: syscall fork -> 15
8: syscall fork -> 16
7: syscall fork -> 17
8: syscall fork -> 18
8: syscall fork -> 19
7: syscall fork -> 20
9: syscall fork -> 21
7: syscall fork -> 22
8: syscall fork -> 23
9: syscall fork -> 24
9: syscall fork -> 25
7: syscall fork -> 26
8: syscall fork -> 27
7: syscall fork -> 28
8: syscall fork -> 29
8: syscall fork -> 30
8: syscall fork -> 31
7: syscall fork -> 32
8: syscall fork -> 33
7: syscall fork -> 34
8: syscall fork -> 35
8: syscall fork -> 36
7: syscall fork -> 37
8: syscall fork -> 38
7: syscall fork -> 39
8: syscall fork -> 40
7: syscall fork -> 41
8: syscall fork -> 42
7: syscall fork -> 43
8: syscall fork -> 44
7: syscall fork -> 45
7: syscall fork -> 46
8: syscall fork -> 47
9: syscall fork -> 48
7: syscall fork -> 49
8: syscall fork -> 50
7: syscall fork -> 51
7: syscall fork -> 52
8: syscall fork -> 53
7: syscall fork -> 54
8: syscall fork -> 55
7: syscall fork -> 56
9: syscall fork -> 57
7: syscall fork -> 58
7: syscall fork -> 59
8: syscall fork -> 60
7: syscall fork -> 61
8: syscall fork -> 62
7: syscall fork -> 63
8: syscall fork -> 64
7: syscall fork -> 65
```

```
8: syscall fork -> 66
7: syscall fork -> -1
8: syscall fork -> -1
9: syscall fork -> -1
OK
ALL TESTS PASSED
```

PID 5 (`usertests`) spawns PID 6, and child processes (PIDs 7, 8, and 9) inherit `trace_mask = 2` from their parent without calling `trace()`, collectively forking PIDs 6 through 66 (61 successful `fork()` calls) until all 64 slots in `proc[NPROC]` are occupied and `allocproc()` returns 0, producing three `syscall fork -> -1` failures before cleanup completes with `OK` and `ALL TESTS PASSED`.

4.4 Console Output Interleaving: `trace 65536 echo hi`

```
$ trace 65536 echo hi
hi67: syscall write -> 2

67: syscall write -> 1
$
```

Mask `65536` (2^{16}) traces `SYS_write = 16`. Inspecting `user/echo.c` confirms that `echo hi` performs two separate `write` system calls: `write(1, "hi", 2)` followed by `write(1, "\n", 1)`. Because `syscall()` calls `printk()` *after* `syscalls[SYS_write]()` finishes:

1. During the first call, the UART console receives `"hi"` (with no newline), and immediately upon return `printk()` appends `67: syscall write -> 2\n` on the same line.

2. Next, `echo` executes `write(1, "\n", 1)`, outputting the newline (seen as the blank line), immediately followed by `67: syscall write -> 1\n`.

4.5 Answers to Observation Questions (`notes.txt`)

1. **Why does `trace 32 grep hello README` show only `read`, and not `open` or `close`?**
   Mask `32` equals 2^5, so only bit 5 (`SYS_read = 5`) is set. `SYS_open = 15` (bit 15) and `SYS_close = 21` (bit 21) both test false under `(p->trace_mask >> num) & 1`.

2. **In `forkforkfork`, multiple different PIDs print lines. Which line of code from Part 2 makes that possible?**
   `np->trace_mask = p->trace_mask;` inside `kfork()` in `kernel/proc.c` (line 291), which copies the parent's mask into each newly allocated child process.

3. **In `trace 65536 echo hi`, why is the trace line printed between `"hi"` and the new line? How many times does `echo` call `write`?**
   `echo` calls `write` **2** times (`write(1, "hi", 2)` and `write(1, "\n", 1)`). Since `syscall()` invokes `printk()` after `syscalls[num]()` executes, `"hi"` reaches the console before `67: syscall write -> 2\n`, while the trailing `"\n"` arrives in the second `write` call before `67: syscall write -> 1\n`.

4. **Which mask traces both `open` (15) and `close` (21), and nothing else?** Setting bits 15 and 21 yields:

$$(1 \ll 15) \mid (1 \ll 21) = 2^{15} + 2^{21} = 32768 + 2097152 = \mathbf{2129920} \quad (\texttt{0x208000}).$$

5 Conclusions

1. **Complete User–Kernel Plumbing**: Implementing `trace(int)` connected all layers of the xv6 system call interface—from `SYS_trace = 23` and RISC-V `ecall` stubs (`user/usys.pl`) through `user/trace.c`, `struct proc`, and dispatch inside `kernel/syscall.c`.

2. **Inheritance Across `fork()` and `exec()`**: Storing `trace_mask` inside `struct proc` preserves tracing across `exec()`, while copying `trace_mask` into the child struct inside `kfork()` automatically propagates tracing across multi-process workloads such as `usertests` (`forkforkfork`).

3. **Post-Syscall Logging Semantics**: Printing right after `syscalls[num]()` writes `p->trapframe->a0` captures both normal returns (byte counts, `argc`) and negative errors (`-1` via `%ld`), while explaining why non-returning calls like `exit` do not log and why `echo`'s two `write` calls interleave with console output.

4. **Bitmask Filtering Efficiency**: Testing `(p->trace_mask >> num) & 1` incurs negligible $O(1)$ overhead per system call while enabling arbitrary syscall subsets (such as `2`, `32`, `65536`, `2129920`, or `2147483647`) dynamically.